Academic Session 2025–26

**EVEN Semester Jan–Jun 2026**

# UNIVERSITY INSTITUTE OF ENGINEERING

## DEPARTMENT AIT-CSE CORE AIML

**Program Code**: AI201

Semester: 4th

**Full Stack - II (23CSH-382)**

# EXPERIMENT 7

## API Integration and Backend Communication

**Practical Session — CO3 - BT3**

**Mr. Prince Pal Singh (E18505)** **Assistant Professor**

# Experiment Objectives

**Learning Outcomes (CO3 - BT3):**

1. **Understand** REST APIs and HTTP methods
2. **Fetch** data from external APIs
3. **Handle** API errors and loading states
4. **Implement** error boundaries
5. **Use** Axios for HTTP requests
6. **Work** with authentication tokens
7. **Build** CRUD operations with backend
8. **Integrate** real backend services

**Duration:** 4-5 hours — **Tools:** Fetch API, Axios, Backend Server

# REST API Fundamentals

**What is REST API?**

- Representational State Transfer
- Uses HTTP for communication
- Stateless and scalable
- Standard way to connect frontend and backend

**HTTP Methods:**

- **GET**: Retrieve data
- **POST**: Create new resource
- **PUT**: Update entire resource
- **PATCH**: Partial update
- **DELETE**: Remove resource

**Status Codes:**

- 2xx: Success (200, 201, 204)
- 4xx: Client error (400, 401, 404)

# Using Fetch API

```
async function getPosts() {
  try {
    const response = await fetch(
      'https://jsonplaceholder.typicode.com/posts'
    );

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}
    }

    const data = await response.json();
    console.log(data);
    return data;
  } catch (error) {
    console.error('Fetch error:', error);
  }
}
```

# Fetch Data in Component

```
import { useState, useEffect } from 'react';

function PostsList() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(data => {
        setPosts(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
```

# POST Request with Fetch

```
async function createPost(title, body) {
  try {
    const response = await fetch(
      'https://jsonplaceholder.typicode.com/posts',
      {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ title, body, userId: 1 })
      }
    );

    if (!response.ok) throw new Error('Failed to create');

    const newPost = await response.json();
    console.log('Created:', newPost);
```

# Axios Installation

**Install:**

```
$ npm install axios
```

**Basic GET request:**

```
import axios from 'axios';

async function getPosts() {
  try {
    const response = await axios.get(
      'https://jsonplaceholder.typicode.com/posts'
    );
    console.log(response.data);
  } catch (error) {
    console.error('Error:', error.response?.data);
```

# CRUD with Axios

```
import axios from 'axios';

const API_URL = 'https://jsonplaceholder.typicode.com/posts';

// CREATE
const createPost = (title, body) =>
  axios.post(API_URL, { title, body, userId: 1 });

// READ
const getPost = (id) =>
  axios.get(`${API_URL}/${id}`);

const getPosts = () =>
  axios.get(API_URL);

// UPDATE
const updatePost = (id, title, body) =>
  axios.put(`${API_URL}/${id}`, { title, body, userId: 1 });

// DELETE
const deletePost = (id) =>
  axios.delete(`${API_URL}/${id}`);

// Usage
getPosts().then(res => console.log(res.data));
createPost('Title', 'Body').then(res => console.log(res.data));
```

# Error Handling

```
async function fetchWithErrorHandling() {
  try {
    const response = await axios.get('/api/data');
    return response.data;
  } catch (error) {
    if (error.response) {
      // Server responded with error status
      console.log('Status:', error.response.status);
      console.log('Data:', error.response.data);
    } else if (error.request) {
      // Request made but no response
      console.log('No response received');
    } else {
      // Error in request setup
      console.log('Error:', error.message);
    }
    throw error;
```

# Axios Interceptors

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'https://api.example.com'
});

// Request interceptor
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Handle unauthorized
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
```

# JWT Authentication

```
import axios from 'axios';

async function login(email, password) {
  try {
    const response = await axios.post('/api/login', {
      email,
      password
    });

    const { token } = response.data;
    localStorage.setItem('token', token);

    // Set default header
    axios.defaults.headers.common['Authorization'] =
      `Bearer ${token}`;

    return response.data;
```

# Custom useFetch Hook

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await axios.get(url);
        setData(response.data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchData();
  }, [url]);

  return { data, loading, error };
}

// Usage in component
function App() {
```

# Loading and Error States

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    axios.get(`/api/users/${userId}`)
      .then(res => {
        setUser(res.data);
        setError(null);
      })
      .catch(err => {
        setError(err.message);
        setUser(null);
      })
      .finally(() => setLoading(false));
```

# Cancel Axios Requests

```
import { useEffect, useRef } from 'react';
import axios from 'axios';

function SearchUsers({ query }) {
  const cancelTokenRef = useRef(null);

  useEffect(() => {
    cancelTokenRef.current = axios.CancelToken.source();

    if (!query) return;

    axios.get(`/api/search?q=${query}`, {
      cancelToken: cancelTokenRef.current.token
    })
      .then(res => console.log(res.data))
      .catch(err => {
        if (axios.isCancel(err)) {
```

# Multiple Parallel Requests

```
async function getDashboardData() {
  try {
    const [users, posts, comments] = await Promise.all([
      axios.get('/api/users'),
      axios.get('/api/posts'),
      axios.get('/api/comments')
    ]);

    console.log(users.data);
    console.log(posts.data);
    console.log(comments.data);

    return {
      users: users.data,
      posts: posts.data,
      comments: comments.data
    },
```

# Submit Form to API

```
import { useState } from 'react';
import axios from 'axios';
import { useForm } from 'react-hook-form';

function CreateUser() {
  const { register, handleSubmit } = useForm();
  const [status, setStatus] = useState('idle');

  const onSubmit = async (data) => {
    setStatus('pending');
    try {
      const response = await axios.post('/api/users', data);
      console.log('User created:', response.data);
      setStatus('success');
    } catch (error) {
      console.error('Error:', error);
      setStatus('error');
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('name')} />
      <input {...register('email')} />
      <button disabled={status === 'pending'}>
        {status === 'pending' ? 'Submitting...' : 'Submit'}
      </button>
      {status === 'success' && <p>User created!</p>}
```

# Retry Failed Requests

```
async function retryRequest(url, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const response = await axios.get(url);
      return response.data;
    } catch (error) {
      if (i === maxRetries - 1) throw error;

      const delay = Math.pow(2, i) * 1000;
      console.log(`Retry ${i + 1} after ${delay}ms`);
      await new Promise(resolve =>
        setTimeout(resolve, delay)
      );
    }
  }
}

// Usage
retryRequest('/api/data').then(data => {
  console.log('Data:', data);
});
```

# Debounce API Calls

```
import { useEffect, useRef } from 'react';

function SearchBox() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const timeoutRef = useRef(null);

  useEffect(() => {
    clearTimeout(timeoutRef.current);

    timeoutRef.current = setTimeout(() => {
      if (query.length > 2) {
        axios.get(`/api/search?q=${query}`)
          .then(res => setResults(res.data));
      }
    }, 500);
```

# File Upload to API

```
import axios from 'axios';

function FileUpload() {
  const [progress, setProgress] = useState(0);

  const handleUpload = async (e) => {
    const file = e.target.files[0];
    const formData = new FormData();
    formData.append('file', file);

    try {
      const response = await axios.post(
        '/api/upload',
        formData,
        {
          headers: {
            'Content-Type': 'multipart/form-data',
```

# Error Boundary Component

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    console.error('Error caught:', error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return (
```

# Mock API for Testing

**Install MSW:**

```
$ npm install msw
```

```
import { rest } from 'msw';
import { setupServer } from 'msw/node';

const server = setupServer(
  rest.get('/api/users', (req, res, ctx) => {
    return res(
      ctx.status(200),
      ctx.json([
        { id: 1, name: 'John' },
        { id: 2, name: 'Jane' }
      ])
    );
  })
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

# API Integration Best Practices

- **Use environment variables**: Store API URLs in .env
- **Error handling**: Always catch and display errors
- **Loading states**: Show feedback during requests
- **Caching**: Avoid duplicate requests
- **Validation**: Validate data before sending
- **Security**: Never expose sensitive data
- **Timeouts**: Set request timeouts
- **Logging**: Log errors for debugging

# Environment Variables

**.env file:**

```
VITE_API_URL=https://api.example.com
VITE_API_KEY=your_api_key_here
```

**Usage:**

```
const API_URL = import.meta.env.VITE_API_URL;
const API_KEY = import.meta.env.VITE_API_KEY;

const api = axios.create({
  baseURL: API_URL,
  headers: {
    'Authorization': `Bearer ${API_KEY}`
  }
});
```

# // SLIDE 24: API SERVICE CLASS

# API Service Class

```
class ApiService {
  constructor(baseURL) {
    this.api = axios.create({ baseURL });
  }

  getUsers = () => this.api.get('/users');
  getUser = (id) => this.api.get(`/users/${id}`);
  createUser = (data) => this.api.post('/users', data);
  updateUser = (id, data) =>
    this.api.put(`/users/${id}`, data);
  deleteUser = (id) => this.api.delete(`/users/${id}`);
}

const apiService = new ApiService(
  import.meta.env.VITE_API_URL
);
```

# Experiment Deliverables

**Build Full Stack CRUD Application:**

**Requirements:**

- React frontend with form handling
- Backend API (Express/Node.js or similar)
- Database integration (MongoDB/MySQL)
- Full CRUD operations
- Authentication with JWT
- Error handling and validation
- Loading states
- Responsive design

**Submission:**

- Frontend GitHub repository
- Backend GitHub repository

# Common API Errors

- **CORS errors**: Enable CORS on backend
- **401 Unauthorized**: Invalid or missing token
- **404 Not Found**: Wrong endpoint URL
- **500 Server Error**: Backend issue
- **Network timeout**: Slow connection or server down
- **JSON parse error**: Invalid response format
- **Missing headers**: Wrong Content-Type
- **Infinite requests**: Missing dependency in useEffect

# API Testing Tools

**Development Tools:**

- **Postman**: Visual API testing
- **Insomnia**: REST client
- **Thunder Client**: VS Code extension
- **curl**: Command line tool

**Test Features:**

- Test all HTTP methods
- Check response status codes
- Validate response data
- Test error scenarios
- Check authentication
- Monitor performance

# Learning Resources

**Documentation:**

- MDN Fetch API docs
- Axios documentation
- REST API best practices
- JWT authentication guide

**Useful APIs:**

- JSONPlaceholder (testing)
- OpenWeatherMap API
- GitHub API
- PokeAPI

# Advanced Topics

- GraphQL alternatives to REST
- WebSocket real-time communication
- Server-sent events (SSE)
- Rate limiting and throttling
- Pagination and filtering
- Search optimization
- API versioning strategies
- Microservices architecture

# Questions & Discussion

*Test your API calls with Postman!*

Monitor network requests in DevTools

# Thank You!

## Experiment 7 Complete

Mr. Prince Pal Singh
prince.18505@cumail.in
Assistant Professor, AIT-CSE

Connect frontend to backend!